

Manual Test Plan

Authentication, Role-Based Access Control and Security

System under test	Nexus RBAC Backend (REST API, Node.js, MongoDB, Redis)
Tool	Postman (any HTTP client works)
Base URL	http://localhost:3000/api/v1
Estimated time	10 minutes
Automated equivalent	docs/TESTING.md (Postman collection and Newman command)

1. Objective

Verify, through the public API, that:

1. Organizations and users can be created, and the password policy is enforced.
2. Role-based access control allows or refuses the same action depending on the caller's role.
3. Repeated failed logins are throttled by a server-side logout.
4. Security-relevant actions are recorded in an admin-only audit log.

2. Preparation

1. Start the stack: `docker compose up --build -d`
2. Open `http://localhost:3000/health/ready`. Expected: `"status": "ready"`.
3. Open Postman and create one new request tab per test case below.

Sending a request in Postman

Action	Where
Choose the method and enter the URL	Request bar
Add a token	Authorization tab, type Bearer Token, paste the token
Add a JSON body	Body tab, raw, format JSON, paste the JSON
Execute	Send; compare the status code with the expected result

Tokens and test data

- A successful login returns an `accessToken`. It identifies the user and role on every later request and is valid for 15 minutes; log in again to obtain a new one.
- Keep the tokens in a notes file under clear names: **ADMIN** and **VIEWER**.
- E-mail addresses in this plan are examples and can be replaced by any address that is not yet registered.
- The example password `N3xus#Secure2026!` satisfies the policy: 8 to 72 characters with upper case, lower case, a digit and a special character.

3. Test cases

TC-01: Register an organization and its admin

Purpose	A new tenant and its first admin are created; the password policy is applied.
Request	POST /auth/register
Auth	none
Expected	201 Created. The response contains user, organization, accessToken and refreshToken. The password is never returned.
Save	accessToken as ADMIN

Request body

```
{
  "organizationName": "Test Company",
  "name": "Alice Admin",
  "email": "alice@testco.com",
  "password": "N3xus#Secure2026!"
}
```

TC-02: List roles

Purpose	The four system roles exist (admin, manager, member, viewer) and an admin may list them.
Request	GET /roles
Auth	ADMIN
Body	none
Expected	200 OK with four roles and their permissions.
Save	The _id of the role named viewer, as VIEWER_ROLE_ID

TC-03: Create a viewer user

Purpose	An admin can add a user to the organization and assign a role.
Request	POST /users
Auth	ADMIN
Expected	201 Created. The new user is returned without a password.

Request body

```
{
  "name": "Victor Viewer",
  "email": "victor@testco.com",
  "password": "N3xus#Secure2026!",
  "roleId": "<VIEWER_ROLE_ID>"
}
```

TC-04: Log in as the viewer

Purpose	Credentials are verified and a short-lived access token plus a rotating refresh token are issued.
Request	POST /auth/login
Auth	none
Expected	200 OK with accessToken and refreshToken.
Save	accessToken as VIEWER

Request body

```
{
  "email": "victor@testco.com",
  "password": "N3xus#Secure2026!"
}
```

TC-05: RBAC: the viewer is refused

Purpose	A read-only role must not be able to create a project.
Request	POST /projects
Auth	VIEWER
Expected	403 Forbidden, error code FORBIDDEN.

Request body

```
{ "name": "Platform Migration" }
```

The caller is authenticated, but the role does not include the project:create permission.

TC-06: RBAC: the admin is allowed

Purpose	The identical request succeeds for a role that includes project:create. Only the token differs from TC-05.
Request	POST /projects
Auth	ADMIN
Expected	201 Created with the new project.

Request body

```
{ "name": "Platform Migration" }
```

TC-07: Brute-force protection

Purpose	Repeated wrong passwords lock the account. The lock is stored server-side and cannot be bypassed by retrying, reloading or opening a new session.
Request	POST /auth/login (send four times in a row)
Auth	none

Request body

```
{
  "email": "victor@testco.com",
  "password": "Wrong#Pass123"
}
```

Attempt	Expected
1	401 Invalid email or password
2	401 Invalid email or password
3	429, code ACCOUNT_LOCKED, retryAfterSeconds (15 seconds)
4 (immediately)	429, still locked

After the lock expires, exactly one further attempt is allowed; if it fails, the account locks again for 15 seconds. A correct login clears the counter.

TC-08: Audit log

Purpose	Security-relevant actions are recorded (actor, action, IP address, user agent, time) and only admins can read them.
Request	GET /audit-logs
Auth	ADMIN
Body	none
Expected	200 OK with events such as org.registered, auth.login and auth.locked.

Permission check: send the same request with the VIEWER token. Expected: 403 Forbidden.

4. Results

Test case	Request	Auth	Expected	Result
TC-01	POST /auth/register	none	201	☐ Pass ☐ Fail
TC-02	GET /roles	ADMIN	200	☐ Pass ☐ Fail
TC-03	POST /users	ADMIN	201	☐ Pass ☐ Fail
TC-04	POST /auth/login	none	200	☐ Pass ☐ Fail
TC-05	POST /projects	VIEWER	403	☐ Pass ☐ Fail
TC-06	POST /projects	ADMIN	201	☐ Pass ☐ Fail
TC-07	POST /auth/login (wrong password, 4x)	none	401, 401, 429 , 429	☐ Pass ☐ Fail
TC-08	GET /audit-logs	ADMIN / VIEWER	200 / 403	☐ Pass ☐ Fail

Acceptance: all eight test cases pass.

Sign-off

Tester	
Date	
Environment	
Overall result	☐ Accepted ☐ Not accepted